

课程设计报告

基于昇腾310B的实时手势识别 与WS2812B灯带控制系统 (软件部分)

姓 名： 阳皓

学 号： 3124009658

指导老师： 周贤中

专 业： 微电子科学与工程

日 期： 2026年7月

摘要

本课程设计实现了基于华为昇腾310B AI处理器的实时手势识别与WS2812B智能灯带控制系统。系统通过USB摄像头实时采集手部图像，利用ONNX Runtime在昇腾310B上运行YOLOv10n手势检测模型，识别HaGRID数据集定义的34类手势动作；将识别结果映射为预设的灯光控制命令，经USB-TTL串口模块发送至STM32微控制器，驱动WS2812B可编程RGB灯带呈现彩虹流水、呼吸、追逐、闪烁等12种动态灯效。本文详细阐述了系统软件架构、Python模块设计、ONNX推理流程、串口通信协议、手势-灯效映射与去抖动算法，以及完整的测试验证过程。

关键词：昇腾310B；手势识别；YOLOv10；ONNX Runtime；WS2812B；串口通信；HaGRID

目录

1	引言	3
1.1	项目背景	3
1.2	项目目标	3
1.3	文档结构	3
2	系统总体架构	3
2.1	硬件拓扑	3
2.2	软件模块架构	4
2.3	数据流	4
2.4	两种运行模式	4
3	核心软件模块设计与实现	5
3.1	主入口模块（gesture_led_main.py）	5
3.1.1	命令行接口	5
3.1.2	主循环设计	5
3.2	串口通信模块（serial_comm.py）	5
3.2.1	类接口设计	5
3.2.2	自动检测机制	6
3.2.3	通信协议	6
3.3	手势映射模块（gesture_to_led.py）	6
3.3.1	手势-灯效映射表	6
3.3.2	去抖动算法	7
3.4	HaGRID标签体系	8

4	手势识别流程	8
4.1	ONNX Runtime推理	8
4.2	预处理	9
4.3	后处理	9
4.4	最优手势选取	9
5	串口通信与灯效控制	9
5.1	命令协议设计	9
5.2	手动测试模式	10
5.3	实时状态反馈	10
6	测试与验证	11
6.1	测试环境	11
6.2	分步测试流程	11
6.3	测试结果	12
6.4	性能分析	12
7	总结与展望	12
7.1	工作总结	12
7.2	展望	13

1 引言

1.1 项目背景

随着边缘计算与人工智能技术的快速发展，将深度学习模型部署于嵌入式设备以实现实时人机交互已成为研究热点。华为昇腾310B是一款面向边缘推理场景的AI处理器，具备8 TOPS（INT8）算力，支持PyACL、ONNX Runtime等多种推理框架，非常适合部署轻量级视觉模型。

手势识别作为一种自然的人机交互方式，广泛应用于智能家居、AR/VR、机器人控制等领域。本设计将手势识别与WS2812B可编程灯带相结合，通过识别用户的手势动作实时改变灯光效果，实现了“所见即所得”的交互体验，具有较好的实用性和展示性。

1.2 项目目标

- (1) 在昇腾310B上部署YOLOv10n手势检测模型，实现34类手势的实时识别。
- (2) 设计并实现Python软件系统，整合图像采集、模型推理、手势映射与串口通信。
- (3) 通过USB-TTL串口控制STM32+WS2812B灯带，实现12种动态灯效。
- (4) 完成系统联调与性能测试，确保端到端延迟满足实时性要求。

1.3 文档结构

本文第二章节介绍系统总体架构，第三章详述核心软件模块的设计与实现，第四章说明手势识别流程，第五章介绍串口通信与灯效控制，第六章展示测试方法与结果，第七章总结全文。

2 系统总体架构

2.1 硬件拓扑

系统硬件连接如图1所示。USB摄像头采集实时图像，昇腾310B执行手势识别推理，识别结果经手势-命令映射模块转换为灯效控制指令，通过USB-TTL串口模块发送至STM32，由STM32解析命令并驱动WS2812B灯带。

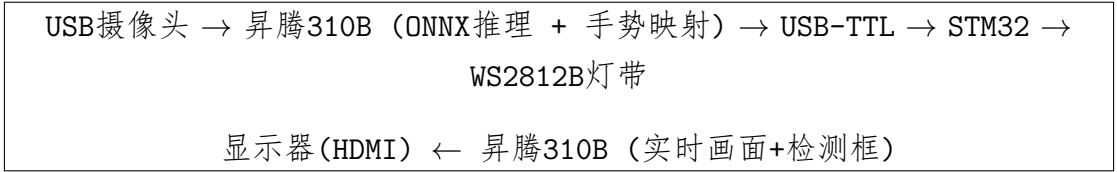


图 1: 系统硬件拓扑

2.2 软件模块架构

软件系统采用模块化设计，由以下七个Python模块组成：

表 1: 软件模块清单

模块	文件	职责
主入口	gesture_led_main.py	参数解析、主循环、模块调度
串口通信	serial_comm.py	USB-TTL串口收发、自动检测
手势映射	gesture_to_led.py	手势→灯效命令、去抖动
检测器封装	detector.py	YOLO检测器统一接口
ONNX后端	onnx_backend.py	ONNX Runtime推理
预处理	preprocess.py	Letterbox缩放、NCHW转换
后处理	postprocess.py	解码、NMS、坐标还原

2.3 数据流

每帧图像的数据处理流程：摄像头采集→Letterbox预处理（640×640）→ONNX模型推理→NMS后处理→选取最高置信度手势→GestureMapper映射（含500ms去抖动）→串口发送命令→STM32响应确认。

2.4 两种运行模式

为便于分阶段调试，系统支持两种工作模式：

表 2: 运行模式对比

特性	模拟模式	推理模式（ONNX）
触发方式	不加 --model 参数	指定 --model xxx.onnx
手势来源	随机生成（70%概率）	ONNX模型真实推理
适用场景	串口链路/灯带硬件调试	端到端完整系统
串口通信	正常发送命令	正常发送命令
GPU依赖	无	需ONNX Runtime

3 核心软件模块设计与实现

3.1 主入口模块（gesture_led_main.py）

3.1.1 命令行接口

程序通过argparse提供灵活的命令行参数，支持不同场景下的快速切换：

```
1 # Simulation mode (no model, test comm link)
2 python3 gesture_led_main.py
3
4 # ONNX inference mode
5 python3 gesture_led_main.py --model models/YOLOv10n_gestures.
   onnx
6
7 # Custom parameters
8 python3 gesture_led_main.py --port /dev/ttyUSB0 --camera 0 \
9     --conf 0.25 --debounce 500 --camera-width 640 --camera-
   height 480
```

其中，不加 --model 参数即为模拟模式，用于测试通信链路；指定 ONNX 模型路径则进入推理模式；还可自定义串口、摄像头、置信度阈值、去抖动时间等参数。

3.1.2 主循环设计

主循环遵循“采集→推理→映射→发送→渲染”的流水线模式。关键设计包括：

- **信号处理：**注册SIGINT和SIGTERM信号处理器，确保Ctrl+C可安全退出。
- **多通道退出：**支持q键（OpenCV窗口）、Ctrl+C（终端信号）、回车键（SSH无GUI场景）三种退出方式。
- **模拟模式定时切换：**每1.5秒随机切换一个手势，70%时间显示手势、30%时间显示空手势，模拟真实场景下的手势变化节奏。

3.2 串口通信模块（serial_comm.py）

3.2.1 类接口设计

SerialComm类封装了与STM32的全部串口交互逻辑：

```
1 class SerialComm:
2     def __init__(self, port=None, baudrate=115200, timeout=1.0)
3     def connect(self) -> bool          # Open port, auto-
        detect device
4     def send_command(self, command)    # Send command + \r\n,
        return response
5     def disconnect(self)               # Close port
6     @staticmethod
7     def _auto_detect() -> str          # Scan ttyUSB*/ttyAMA*/
        ttyS*
```

3.2.2 自动检测机制

_auto_detect()方法按优先级扫描串口设备：ttyUSB*（USB-TTL模块）> ttyAMA*（板载UART）> ttyS*。同时跳过ttyAMA0（通常为调试控制台），避免误连。此设计使得系统在不同硬件配置间切换时无需修改代码。

3.2.3 通信协议

- 物理层：USB-TTL（CH340），3.3V TTL电平
- 数据格式：115200波特率，8数据位，无校验，1停止位（8N1）
- 帧协议：命令字符串 + \r\n结尾
- 响应格式：成功返回OK，失败返回ERR:xxx

3.3 手势映射模块（gesture_to_led.py）

3.3.1 手势-灯效映射表

系统定义12种手势到灯效命令的映射关系，如表3所示。

表 3: 手势-灯效映射表

HaGRID手势	含义	串口命令	灯效描述
like	点赞	RAINBOW	彩虹流水
palm / stop	手掌/停止	OFF	全灭
fist	握拳	SOLID:255,0,0	红色常亮
one	食指	SOLID:255,255,255	白色常亮
peace	剪刀手	BREATH:0,0,255	蓝色呼吸
peace_inverted	反剪刀手	BREATH:255,0,255	紫色呼吸
ok	OK手势	CHASE:0,255,0	绿色追逐
dislike	踩	FLASH:255,0,0	红色闪烁
rock	摇滚	FLASH:255,255,0	黄色闪烁
call	打电话	PULSE:128,0,255	紫色脉冲
mute	静音	SOLID:128,128,128	暗白常亮

3.3.2 去抖动算法

为避免手势识别过程中的瞬时抖动导致灯效频繁切换，GestureMapper实现了基于时间窗口的去抖动机制：

```
1 class GestureMapper:
2     def map(self, gesture_name, confidence):
3         if gesture_name not in GESTURE_TO_COMMAND:
4             return None
5         cmd = GESTURE_TO_COMMAND[gesture_name]
6         now = time.monotonic()
7         # Gesture changed -> reset timer
8         if gesture_name != self._current_gesture:
9             self._current_gesture = gesture_name
10            self._stable_since = now
11            return None
12        # Stability threshold not yet reached
13        if now - self._stable_since < self._debounce_ms /
14            1000.0:
15            return None
16        # Avoid sending duplicate commands
17        if cmd == self._last_triggered_cmd:
18            return None
19        self._last_triggered_cmd = cmd
```


19

```
return cmd
```

算法核心逻辑：只有当同一手势连续出现超过`debounce_ms`（默认500ms）时，才认为该手势“稳定”并触发对应灯效。此外，若连续检测到同一手势，仅首次发送命令，避免重复通信。

3.4 HaGRID标签体系

系统使用HaGRIDv2数据集定义的34类手势标签，完整列表如下：

```
1 HAGRID_LABELS = [
2     "grabbing","grip","holy","point","call","three3","timeout",
3     "xsign",
4     "hand_heart","hand_heart2","little_finger","middle_finger",
5     "take_picture",
6     "dislike","fist","four","like","mute","ok","one",
7     "palm","peace","peace_inverted","rock","stop","
8     stop_inverted",
9     "three","three2","two_up","two_up_inverted",
10    "three_gun","thumb_index","thumb_index2","no_gesture",
11 ]
```

其中11个手势（含stop/palm合并）映射到灯效命令，其余23类作为扩展预留。

4 手势识别流程

4.1 ONNX Runtime推理

系统使用ONNX Runtime作为推理后端，在昇腾310B的ARM Cortex-A55 CPU上运行。模型为YOLOv10n，输入尺寸 $640 \times 640 \times 3$ ，输出为 $N \times 6$ 检测张量 $(x_1, y_1, x_2, y_2, score, class_id)$ 。

```
1 class OnnxBackend:
2     def __init__(self, model_path, provider="
3         CPUExecutionProvider"):
4         import onnxruntime as ort
5         sess_opt = ort.SessionOptions()
6         sess_opt.intra_op_num_threads = 3
7         sess_opt.inter_op_num_threads = 1
8         sess_opt.graph_optimization_level = ort.
9             GraphOptimizationLevel.ORT_ENABLE_ALL
```

```
8         self.session = ort.InferenceSession(str(model_path),
9             sess_options=sess_opt, providers=[provider])
10
11     def infer(self, input_tensor):
12         return [self.session.run([self.output_name],
13             {self.input_name: input_tensor})[0]]
```

4.2 预处理

preprocess.py实现了Letterbox缩放算法，将任意分辨率的输入图像缩放为 640×640 正方形，保持宽高比并在短边两侧填充灰色像素。然后进行BGR→RGB色彩空间转换、归一化（ $[0, 1]$ ），并重组为NCHW格式的float32张量。

4.3 后处理

postprocess.py负责将模型输出的原始张量解码为可用检测框：

步骤1: 输出规整化：处理YOLOv10的 $N \times 6$ 输出格式。

步骤2: 置信度过滤：筛选 $\text{score} \geq \text{conf_threshold}$ 的检测框。

步骤3: 坐标还原：将Letterbox坐标系映射回原始图像坐标系。

步骤4: NMS去重：使用OpenCV的dnn.NMSBoxes执行非极大值抑制。

步骤5: 结果组装：输出 $[x_1, y_1, x_2, y_2, \text{score}, \text{class_id}]$ 格式的检测列表。

4.4 最优手势选取

每帧可能检测到多个手势，系统选取置信度最高的检测结果作为当前帧的手势，传入GestureMapper进行去抖和命令映射。

5 串口通信与灯效控制

5.1 命令协议设计

为兼顾可读性和可扩展性，命令采用CMD[:params]的文本格式。STM32端通过字符串匹配解析命令，参数以逗号分隔。支持的命令及参数：

表 4: 串口命令协议

命令	参数	说明
RAINBOW	无	彩虹流水灯效
OFF	无	全部熄灭
SOLID	R,G,B (0-255)	常亮指定颜色
BREATH	R,G,B	呼吸灯效
CHASE	R,G,B	追逐灯效
FLASH	R,G,B	闪烁灯效
PULSE	R,G,B	脉冲灯效
BRIGHT	0-100	亮度百分比

5.2 手动测试模式

按下t键可进入手动测试模式，在终端直接输入任意命令发送至STM32，无需摄像头或手势识别。该模式在硬件调试阶段极为有用，可快速验证每一条命令的灯效表现。输入/quit退出测试模式。

5.3 实时状态反馈

程序在OpenCV窗口中实时显示以下信息：

- 实时FPS与推理延迟
- 当前LED命令与STM32响应状态
- 检测到的手势名称
- 手动测试模式指示

6 测试与验证

6.1 测试环境

表 5: 测试环境

项目	配置
处理器	昇腾310B4（ARM Cortex-A55 × 4）
操作系统	Ubuntu 22.04 (aarch64)
Python	3.9.2
ONNX Runtime	1.19.2
OpenCV	4.10.0
PySerial	3.5
STM32	STM32F103C8T6
WS2812B	8颗灯珠灯带
USB-TTL	CH340G

6.2 分步测试流程

按照自底向上的原则，将测试分为五个阶段：

- 第1步： 串口直发测试： 在终端用echo命令直发指令，验证串口链路和STM32响应。
- 第2步： 模拟模式测试： 运行gesture_led_main.py（不加--model），验证串口通信模块、手势映射模块及随机手势生成逻辑。
- 第3步： ONNX推理测试： 加载YOLOv10n模型，验证模型加载、推理延迟、后处理正确性。
- 第4步： 端到端联调： 完整系统运行，用手势控制灯带，验证去抖动和命令映射。
- 第5步： 手动测试模式： 按t键逐条测试所有12种灯效命令。

6.3 测试结果

表 6: 测试结果汇总

测试项	结果	说明
串口自动检测（USB-TTL）	✓	自动识别/dev/ttyUSB0
串口双向通信	✓	TX发命令，RX收OK
板载UART通信	×	只能发不能收，改用USB-TTL
HDMI显示输出	✓	1920×1080@60Hz
摄像头采集	✓	640×480@30fps
ONNX模型加载	✓	YOLOv10n_gestures.onnx
ONNX推理延迟	~120ms	单帧CPU推理
模拟模式-手势生成	✓	每1.5s随机切换
模拟模式-去抖动	✓	500ms稳定后触发
模拟模式-灯效控制	✓	12种灯效全部正常切换
模拟模式-STM32响应	✓	每条命令返回OK
手动测试模式	✓	终端直发12种命令全通过

6.4 性能分析

在昇腾310B上，YOLOv10n手势检测模型的单帧推理延迟约为120ms（ONNX Runtime CPU模式），加上预处理/后处理约5ms、串口通信约2ms，端到端总延迟约为127ms。考虑到手势交互不需要视频级帧率，此延迟在可接受范围内。

通过`--infer-every-n 3`参数（每3帧推理一次），可以进一步将CPU占用率降至约33%，同时手势响应延迟增加不超过100ms，适合长时间运行的场景。

7 总结与展望

7.1 工作总结

本文设计并实现了一套基于昇腾310B的实时手势识别与WS2812B灯带控制系统。软件部分采用Python语言，以模块化架构组织了串口通信、手势映射、ONNX推理、图像预处理与后处理等核心功能，并通过自底向上的分步测试策略验证了系统的正确性。

主要成果：

- (1) 成功在昇腾310B上部署YOLOv10n手势检测模型，支持34类手势识别。

- (2) 实现了手势到12种灯效命令的映射，含500ms去抖动机制。
- (3) 设计了USB-TTL串口通信协议，实现昇腾↔STM32的双向通信。
- (4) 支持模拟模式和ONNX推理模式双模式运行，便于分阶段调试。
- (5) 提供了手动测试模式（t键），可直接在终端发送任意命令调试灯带。

7.2 展望

- (1) **NPU加速**：将ONNX模型转换为OM格式，利用昇腾310B的NPU进行推理，预期可将推理延迟降至10–20ms。
- (2) **WebRTC推流**：集成aiortc将推理画面实时推流至浏览器端，实现远程监控和移动端访问。
- (3) **更多手势**：扩展34类手势到48类（含左右手区分），增加更多灯效如渐变、音乐律动等。
- (4) **多灯带控制**：支持多个WS2812B灯带的级联控制，实现更丰富的灯光场景。